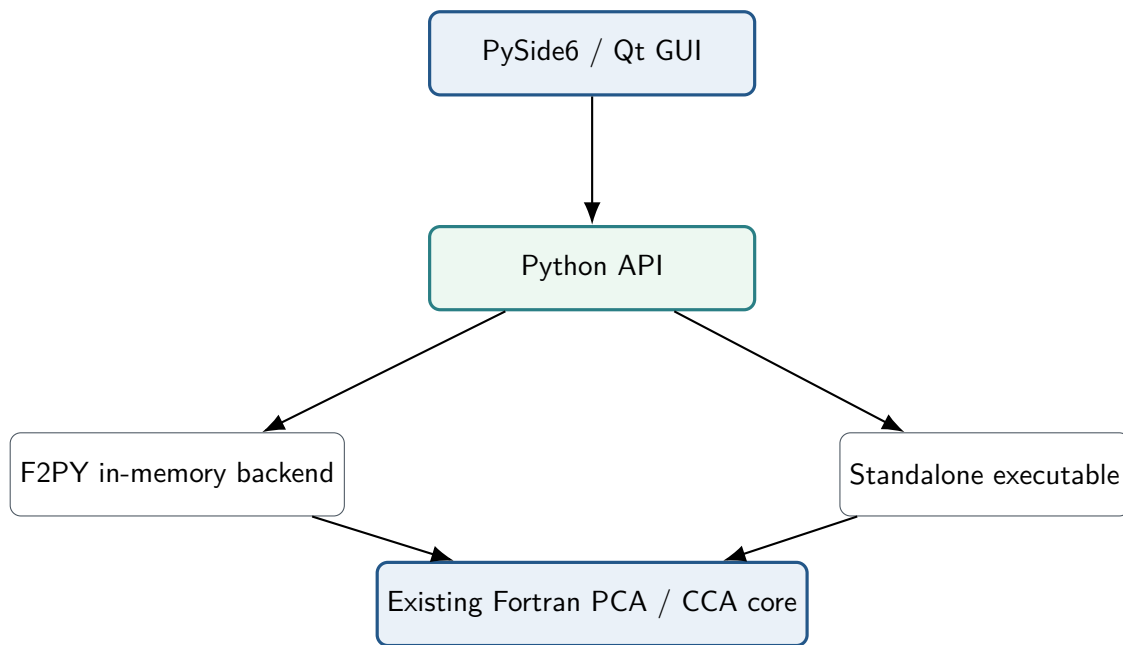

FracVAL-Qt 1.0.1

User and Developer Guide

Fortran fractal aggregate engine, Python API, and native Qt desktop GUI



FracVAL-Qt preserves the established FracVAL numerical aggregation routines and adds runtime configuration, reproducible random seeds, a Python interface, an optional intended-contact overlap model, a native desktop GUI, 3-D visualization, result export, and regression tests. The original FracVAL software and methodology were developed by J. Morán, A. Fuentes, F. Liu, and J. Yon. FracVAL-Qt is a GPLv3 derivative and does not imply endorsement of later modifications by the original authors.

Contents

1	What FracVAL Is and Which Interface to Use	1
1.1	Origin, attribution, citation, and license	1
1.2	Three supported workflows	2
1.3	What changed relative to the legacy source layout	2
2	Project Structure	4
2.1	Source directories	5
2.1.1	src/: scientific engine	5
2.1.2	python/fracval/: public Python library	5
2.1.3	python/fracval/desktop/: native GUI	5
2.2	Generated directories	5
3	Installation and First Build	6
3.1	Requirements	6
3.2	All platforms: conda/Miniforge	6
3.3	macOS and Linux without conda	7
3.4	Windows notes	7
3.5	Installation targets	7
3.6	Verify the installation	8
4	Five-Minute Workflows	9
4.1	Launch the GUI	9
4.2	Generate from Python	9
4.3	Run the standalone executable	9
5	Native Qt GUI Guide	11
5.1	Window organization	11
5.2	Aggregate parameters	11
5.3	Random seeds	12
5.4	Contact overlap	12
5.5	Advanced controls	12
5.6	Generation, progress, and cancellation	13
5.7	Display and appearance	13
5.8	Statistics	13
5.9	Saving and loading	13
6	Python API	15

6.1	Import surface	15
6.2	Configuration object	15
6.2.1	Configuration JSON	16
6.3	Single aggregate generation	16
6.4	The Aggregate object	16
6.5	Batch generation	17
6.6	Result bundles	17
6.7	Interactive visualization	18
6.8	Error handling	18
6.9	Diagnostics from Python	19
7	Embedding FracVAL in Another Python Codebase	20
7.1	Recommended boundary	20
7.2	Why this boundary matters	21
7.3	Parallel workloads	21
7.4	Adding Python-only post-processing	21
8	Standalone Fortran Runtime Interface	22
8.1	The namelist file	22
8.2	Input parameter reference	22
8.3	Run multiple cases without recompilation	23
9	Model and Numerical Behavior	24
9.1	Fractal target relation	24
9.2	Primary-particle size distribution	24
9.3	Construction restarts	24
10	Intended Contact Overlap	25
10.1	Purpose	25
10.2	Fixed overlap	25
10.3	Statistical overlap	26
10.4	Collision protection	26
10.5	Recorded contact history	26
11	Output Formats and Provenance	27
11.1	Native DAT	27
11.2	CSV	27
11.3	Contact CSV	27
11.4	XYZ	27
11.5	Metadata JSON	27
11.6	Interactive HTML	28
12	Visualization Tools	29
12.1	Standalone plotting script	29
12.2	Notebooks	29
12.3	Performance considerations	29
13	Backends and Code Architecture	30
13.1	Call graph	30

13.2 F2PY extension backend	30
13.3 Executable fallback	31
13.4 Why both backends exist	31
14 Testing and Validation	32
14.1 Core test command	32
14.2 What the tests cover	32
14.3 Qt tests	33
14.4 Debug Fortran build	33
15 Extending FracVAL Safely	34
15.1 Python-only feature	34
15.2 Adding a new physical generation parameter	34
15.3 Adding a new export format	34
15.4 Adding another visualization backend	35
15.5 Version-control guidance	35
16 Troubleshooting	36
16.1 No backend is available	36
16.2 Could not find the Qt platform plugin "cocoa"	36
16.3 F2PY extension build fails	36
16.4 Windows: ImportError: DLL load failed for the extension	37
16.5 Many PC not able to continue / restart messages	37
16.6 The GUI opens but generation fails	37
16.7 3-D rendering is slow	37
17 Command and API Reference	38
17.1 Installed commands	38
17.2 Public Python functions	38
17.3 FracVALConfig fields	39
17.4 ViewerAppearance fields	39
18 Worked Python Examples	40
18.1 Basic generation and bundle export	40
18.2 Overlap comparison	40
18.3 Batch generation	41
18.4 Parameter sweep	41
A Makefile Target Reference	43
B Recommended Scientific Workflow Checklist	44

Chapter 1

What FracVAL Is and Which Interface to Use

FRACVAL generates aggregates composed of spherical primary particles. The numerical core uses particle-cluster aggregation (PCA) and cluster-cluster aggregation (CCA) routines. In this distribution, those Fortran routines remain the scientific engine while configuration, scripting, visualization, and interactive use are handled by separate layers.

The main design goal is that changing a simulation parameter does *not* require recompiling the Fortran code. Recompilation is only necessary after source-code changes or when building the native Python extension for a new Python/platform environment.

1.1 Origin, attribution, citation, and license

FracVAL-Qt is a derivative and modernization of the original FRACVAL Fortran software developed by J. Morán, A. Fuentes, F. Liu, and J. Yon. The original scientific work is:

J. Morán, A. Fuentes, F. Liu, and J. Yon, “FracVAL: An improved tunable algorithm of cluster-cluster aggregation for generation of fractal structures formed by polydisperse primary particles,” *Computer Physics Communications*, 239 (2019), 225–237. DOI: [10.1016/j.cpc.2019.01.015](https://doi.org/10.1016/j.cpc.2019.01.015).

The corresponding original software/data release is archived at Mendeley Data, Version 1, DOI: [10.17632/mgf8wdcsfb.1](https://doi.org/10.17632/mgf8wdcsfb.1). The original release specifies the GNU General Public License version 3. This repository therefore includes the GPLv3 text in `LICENSE` and keeps a plain-language provenance record in `NOTICE.md`.

The original authors should be credited for the original FracVAL algorithm and Fortran implementation. Runtime configuration, the Python/F2PY layer, Qt GUI, visualization, testing infrastructure, reproducibility controls, and overlap extensions described in this manual are subsequent modifications maintained in FracVAL-Qt. Nothing in this distribution implies that the original authors created or endorsed those later changes.

For publications, cite the original Morán et al. paper for the FracVAL methodology. When exact software provenance matters, additionally cite the FracVAL-Qt repository and release/tag used.

Machine-readable citation metadata is provided in `CITATION.cff`; a copy-pasteable BibTeX entry is in `CITATION.md`.

Important

Do not remove original authorship or license notices when redistributing a modified version. FracVAL-Qt is distributed under GPLv3; see `LICENSE` for the complete terms.

1.2 Three supported workflows

Interface	Use it when	Typical entry point
Qt GUI	You want to interactively change parameters, generate one or many aggregates, inspect them in 3-D, change appearance, and export results.	<code>fracval-gui</code>
Python API	You are writing scripts, notebooks, parameter sweeps, optimization loops, pipelines, or another application.	<code>from fracval import FracVALConfig, generate</code>
Fortran CLI	You want a small compiled executable controlled by a namelist file, for example on a compute node.	<code>./build/fracval fracval.in</code>

Recommended practice

For new Python-based scientific work, use the public Python API even if you also use the supplied GUI. Do not make your analysis code depend on GUI widgets. This keeps generation reproducible and makes your workflow easier to test.

1.3 What changed relative to the legacy source layout

The modernization in this package includes:

- runtime input instead of compile-time user parameters;
- out-of-tree Fortran compilation under `build/`;
- deterministic random-seed control;
- an F2PY wrapper that returns particle arrays directly to Python;
- a native PySide6 / Qt desktop interface;
- offline Plotly 3-D visualization embedded in the GUI;
- fixed or bounded statistical overlap at intended joining contacts;

- contact-overlap history and derived geometry in Python;
- portable JSON configuration and result-bundle helpers;
- deterministic mono/polydisperse and overlap regression tests.

Chapter 2

Project Structure

The source tree is intentionally layered so that the GUI, Python API, and Fortran engine can evolve independently.

```
FracVAL-Qt/
|-- Makefile
|-- pyproject.toml
|-- fracval.in
|-- README.md
|-- CHANGELOG.md
|-- LICENSE
|-- NOTICE.md
|-- CITATION.cff
|-- CITATION.md
|-- CONTRIBUTING.md
|-- src/                                # Fortran source
|   |-- Ctes.f90
|   |-- Frac_VAL_CCA.f90
|   |-- CCA_module.f90
|   |-- PCA_Subclusters_module.f90
|   |-- PCA_cca.f90
|   |-- a_Random_PP.f90
|   |-- RAND_SAMPLE.f90
|   |-- random.f90
|   |-- Save_results_CC.f90
|   '-- fracval_python_api.f90
|-- python/
|   '-- fracval/
|       |-- config.py
|       |-- aggregate.py
|       |-- engine.py
|       |-- io.py
|       |-- visualization.py
|       |-- diagnostics.py
|       '-- desktop/
|-- tools/                              # cross-platform build script (tools/build.py)
|-- gui/                                # source-tree GUI launcher
|-- examples/                           # Python integration examples
```



```

|-- tests/          # Fortran, Python, Qt and model tests
|-- plot/           # standalone plots and notebooks
|-- doc/            # this manual and quick references
|-- results/        # default CLI output
'-- build/          # generated native/doc build products

```

2.1 Source directories

2.1.1 `src/`: scientific engine

`src/` contains the Fortran aggregation implementation. The modernized runtime state and namelist parsing live in `Ctes.f90`. The thin wrapper `fracval_python_api.f90` exposes a single aggregate-generation routine to F2PY without moving the PCA/CCA algorithm into Python.

2.1.2 `python/fracval/`: public Python library

This directory is the normal integration boundary for Python users:

<code>config.py</code>	immutable validated runtime configuration;
<code>engine.py</code>	backend selection and single/batch generation;
<code>aggregate.py</code>	aggregate result container and derived geometry;
<code>io.py</code>	native data reading and portable result bundles;
<code>visualization.py</code>	reusable Plotly/Matplotlib visualization;
<code>diagnostics.py</code>	environment and backend diagnostics;
<code>desktop/</code>	Qt-specific application code.

2.1.3 `python/fracval/desktop/`: native GUI

The desktop package contains the Qt main window, background worker, WebEngine viewer, and Qt plugin-path setup. Scientific generation is not implemented in these modules; the GUI calls the same public Python engine used by scripts.

2.2 Generated directories

Compiler products belong under `build/`. A platform-specific F2PY module is generated under `python/fracval/` and removed by `make clean`. The source ZIP intentionally does not carry a Linux or macOS binary extension, because native extensions must match the user's Python version and machine.

Chapter 3

Installation and First Build

3.1 Requirements

Core generation needs:

- Python 3.10 or newer for the Python API;
- NumPy;
- GNU Fortran (`gfortran`);
- a C compiler for the F2PY wrapper;
- GNU Make on macOS/Linux for the optional Makefile shortcuts; Windows uses the Python build script directly.

The native GUI additionally needs PySide6. Plotly is a base Python dependency because it is used by the GUI viewer and public interactive plotting API. Static Matplotlib plotting and notebooks are available through the optional `plot` dependency group.

3.2 All platforms: conda/Miniforge

The supported installation uses a conda-forge environment so the Fortran and C compilers, NumPy, and Python match on every operating system.

macOS and Linux:

```
conda env create -f environment.yml
conda activate fracval
python -m pip install -e "[gui,dev]"
python tools/build.py all
python -m pytest
```

Windows (Miniforge Prompt):

```
conda env create -f environment-windows.yml
conda activate fracval
python -m pip install -e "[gui,dev]"
```

```
python tools\build.py all
python -m pytest
```

`tools/build.py all` compiles the standalone executable and the in-memory F2PY extension. On Windows it uses the MinGW-w64 GNU toolchain provided by the conda-forge packages `gfortran_win-64` and `gcc_win-64`, links the GNU runtime libraries statically, and imports the freshly built extension in a clean subprocess before reporting success. If that import needs the gfortran runtime DLLs, they are copied next to the extension automatically. MSVC is not supported for the extension because Fortran and C objects are linked by gfortran into one module.

Important

Use the same Python interpreter for installing PySide6, building the extension, and launching the GUI. `fracval-info` prints the interpreter, the discovered compilers, and the available backends.

3.3 macOS and Linux without conda

With `gfortran`, a C compiler, and GNU Make from the operating system (Homebrew `gcc`; `apt install gfortran gcc make`), a plain virtual environment and the Makefile shortcuts work:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[gui,dev]"
make
make python-ext PYTHON=python
make test PYTHON=python
```

3.4 Windows notes

- Run commands from the Miniforge Prompt so the MinGW toolchain is on PATH.
- `fracval-info` reports `not found` for a compiler when the environment is incomplete; fix with `conda install -c conda-forge gfortran_win-64 gcc_win-64`.
- The standalone executable is `build\fracval.exe`. Namelist `output_dir` values may use forward or backward slashes.
- WSL2 with the Linux instructions remains supported.

3.5 Installation targets

Command	Purpose
<code>make install</code> <code>PYTHON=python</code>	Editable Python package installation plus local F2PY extension build.
<code>make install-gui</code> <code>PYTHON=python</code>	Same, with PySide6 GUI dependencies.

<code>make</code>	Build the standalone Fortran executable at <code>build/fracval</code> .
<code>python tools/build.py</code>	Build executable and extension on any platform.
<code>all</code>	
<code>make python-ext</code>	Rebuild only the F2PY extension (POSIX shortcut for
<code>PYTHON=python</code>	<code>tools/build.py ext</code>).
<code>python tools/build.py</code>	Remove native build products.
<code>clean</code>	
<code>make clean</code>	Remove native compiler products and extension binaries.

3.6 Verify the installation

Run:

```
fracval-info
```

A healthy installation should report the active Python, package directory, and at least one available backend. For the preferred Python workflow you want:

```
F2PY extension      : available
Available backends  : extension, executable
```

For GUI-specific diagnostics:

```
fracval-qt-check
```

On macOS, the available platform plugins should include `cocoa`. The launcher explicitly configures Qt's platform-plugin path before constructing `QApplication` to avoid the common empty-plugin-path failure. On Windows the expected plugin is `windows`.

Chapter 4

Five-Minute Workflows

4.1 Launch the GUI

After the first build:

```
cd /path/to/FracVAL-Qt
conda activate fracval
fracval-gui
```

You do not run `make` again merely because `N`, `Df`, `kf`, particle size, overlap, or aggregate count changed.

4.2 Generate from Python

```
from fracval import FracVALConfig, generate

config = FracVALConfig(
    n=100,
    df=1.79,
    kf=1.40,
    rp_g=15.0,
    rp_gstd=1.0,
    seed=12345,
)

aggregate = generate(config)
print(aggregate.data.shape)
print(aggregate.radius_of_gyration)
```

4.3 Run the standalone executable

```
make
./build/fracval fracval.in
```

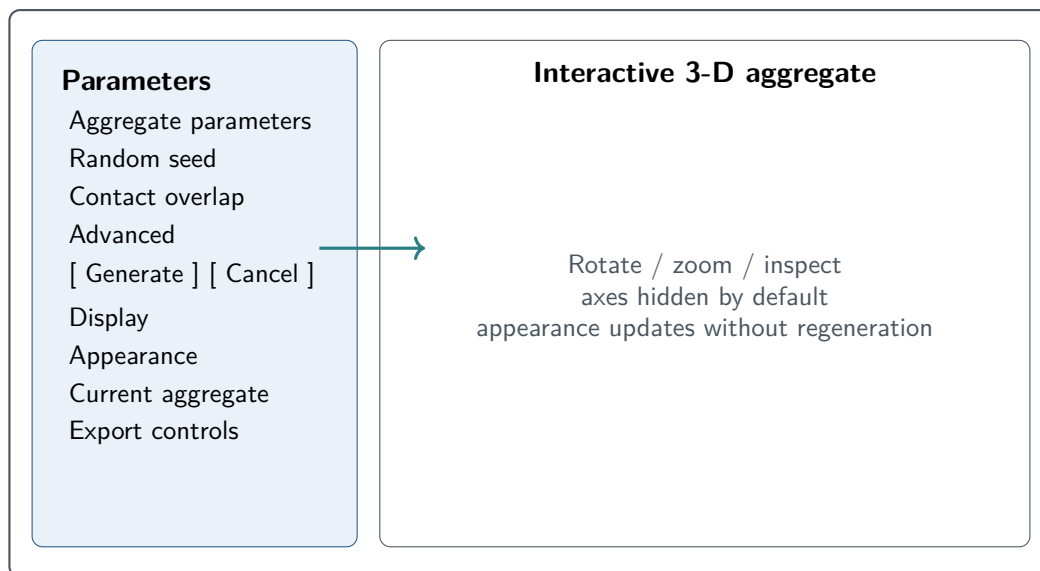
Edit `fracval.in` and rerun the same executable. No recompilation is needed for runtime parameter changes.

Chapter 5

Native Qt GUI Guide

5.1 Window organization

The desktop application uses a scrollable control panel on the left and an interactive 3-D viewer on the right. The layout can be resized with the splitter.



5.2 Aggregate parameters

Primary particles (N)

Number of spheres in each aggregate.

Fractal dimension (Df)

Fractal dimension used by the legacy target-radius relation.

Fractal prefactor (kf)

Fractal prefactor used with Df.

Particle distribution

Monodisperse or polydisperse.

Geometric mean radius

`rp_g`, in the user's chosen length units.

Geometric radius std. dev.

`rp_gstd`. The GUI forces 1.0 in monodisperse mode.

Number of aggregates

Number generated in the current batch.

All coordinates and radii use the same arbitrary length unit. If `rp_g` is interpreted in nanometers, the returned coordinates are also in nanometers. The engine does not attach a unit string.

5.3 Random seeds

With *Use fixed base seed* enabled, the entered base seed makes a single run reproducible and gives a deterministic sequence of per-aggregate seeds for a batch. Disable it for exploratory random generation. The GUI always displays the actual seed stored in each returned aggregate.

Recommended practice

For publication-quality or archived simulation work, use fixed seeds and export JSON metadata or result bundles. The seed should be treated as part of the simulation input.

5.4 Contact overlap

The GUI exposes three modes:

- **None (touching)**: legacy contact behavior;
- **Fixed**: every intended joining contact uses one overlap fraction;
- **Statistical**: each intended contact samples a bounded normal distribution.

The GUI displays overlap values as percentages. The Python API and Fortran namelist use fractions, so 5 percent in the GUI corresponds to 0.05 in Python.

Unrelated particle pairs are still protected by the strict unintended-overlap tolerance. See Chapter 10 for details.

5.5 Advanced controls

The Advanced panel contains:

- `Ext_case`, the legacy extreme-case switch;
- `Nsubcl_perc`, sub-cluster fraction (normally 0.10);
- `tol_ov`, numerical tolerance for unintended overlap;
- maximum construction attempts;
- backend selection (auto, extension, or executable when available).

Unless reproducing a legacy special case, keep `Ext_case=0`. Do not use `tol_ov` as a physical overlap control; use the Contact overlap panel.

5.6 Generation, progress, and cancellation

Click *Generate*. A Qt worker object performs generation outside the GUI event loop. For batches, each completed aggregate is added to the selection box and the progress bar advances.

Cancellation is cooperative *between* aggregates. The legacy Fortran routine cannot safely be interrupted in the middle of one aggregate build, so clicking Cancel does not forcibly terminate the current native routine.

5.7 Display and appearance

The Display panel offers:

- **Spheres:** geometric sphere meshes, best for smaller aggregates;
- **Centers:** lighter marker rendering for large aggregates;
- sphere resolution for the mesh mode.

The Appearance panel controls rendering only; it does not regenerate or modify the physical aggregate. Controls include:

- solid color or color by particle radius;
- particle color and Plotly color scale;
- opacity;
- shininess/specular appearance in sphere mode;
- background color;
- XYZ axes/grid visibility (off by default);
- radius legend and plot title visibility.

5.8 Statistics

For the selected aggregate the GUI shows particle count, seed, backend, construction attempts (when reported), mean radius, contact count, overlap statistics, radius of gyration, and bounding radius.

The Python radius of gyration includes the intrinsic moment contribution of solid spheres (see Section 6.4). This reported quantity is a post-generation geometric metric and should not be confused blindly with the intermediate target radius used inside the legacy placement routines.

5.9 Saving and loading

The File menu can save/load parameter JSON. A GUI parameter file includes:

- the `FracVALConfig` fields;
- batch count;
- backend selection;
- viewer appearance.

The GUI exports the selected aggregate as DAT, CSV, contacts CSV, XYZ, metadata JSON, or a standalone interactive HTML view. A whole batch can be written as a ZIP archive.

Chapter 6

Python API

6.1 Import surface

The supported public interface is exported from `fracval`:

```
from fracval import (
    Aggregate,
    FracVALConfig,
    GenerationError,
    ViewerAppearance,
    generate,
    generate_batch,
    iter_generate_batch,
    load_data,
    save_bundle,
    load_bundle,
    plot_3d,
    plot_static,
    available_backends,
    extension_available,
    runtime_info,
)
```

Downstream code should prefer these names rather than importing private modules or `fracval.desktop` internals.

6.2 Configuration object

`FracVALConfig` is an immutable dataclass. Calling `validate()` checks physical/numerical ranges before invoking Fortran.

```
from fracval import FracVALConfig

cfg = FracVALConfig(
    n=250,
    df=1.80,
```

```

    kf=1.20,
    rp_g=20.0,
    rp_gstd=1.5,
    seed=73001,
    max_attempts=500,
)
cfg.validate()

```

Because it is immutable, derive variants without mutating shared state:

```

case_b = cfg.with_updates(df=1.9, kf=1.35)
case_c = cfg.with_seed(73002)

```

6.2.1 Configuration JSON

```

cfg.save_json("case.json")
restored = FracVALConfig.load_json("case.json")

```

`load_json()` also accepts the richer JSON file saved by the Qt GUI and extracts its nested `config` object.

6.3 Single aggregate generation

```

from fracval import generate

agg = generate(cfg, backend="auto")

```

`backend="auto"` prefers the F2PY extension and falls back to the standalone executable if available. Use `extension` or `executable` when a workflow requires one specifically.

6.4 The Aggregate object

`Aggregate` contains NumPy arrays and provenance:

```

agg.x                # (N,)
agg.y
agg.z
agg.radius
agg.xyz              # (N, 3)
agg.data             # (N, 4): x, y, z, radius
agg.config
agg.seed
agg.backend
agg.attempts

```

The mass weighting used by geometric metrics is proportional to r^3 , so a common material density cancels. The Python center of mass is

$$\mathbf{r}_{\text{cm}} = \frac{\sum_i r_i^3 \mathbf{r}_i}{\sum_i r_i^3}. \quad (6.1)$$

The reported radius of gyration includes the intrinsic moment of each solid sphere:

$$R_g = \left[\frac{\sum_i r_i^3 \left(\|\mathbf{r}_i - \mathbf{r}_{\text{cm}}\|^2 + \frac{3}{5} r_i^2 \right)}{\sum_i r_i^3} \right]^{1/2}. \quad (6.2)$$

The bounding radius is the maximum center distance from the aggregate center of mass plus that particle's radius.

6.5 Batch generation

```
from fracval import generate_batch

items = generate_batch(50, cfg)
```

For long runs, prefer the streaming form:

```
from fracval import iter_generate_batch

for index, agg in enumerate(iter_generate_batch(50, cfg), start=1):
    print(index, agg.seed, agg.radius_of_gyration)
```

A deterministic NumPy random generator derives one seed per aggregate from the resolved base seed. This makes the batch seed sequence repeatable.

6.6 Result bundles

Native `.dat` files are intentionally simple and contain only four columns. For reproducible Python workflows, save metadata alongside particle data:

```
from fracval import save_bundle, load_bundle

paths = save_bundle(agg, "results/case01", stem="aggregate_0001")
restored = load_bundle(paths["json"])
```

By default a bundle writes:

- `aggregate_0001.dat`;
- `aggregate_0001.csv`;
- `aggregate_0001.contacts.csv`;
- `aggregate_0001.json`.

Add XYZ when required:

```
save_bundle(
    agg,
    "results/case01",
    stem="aggregate_0001",
    formats=("dat", "csv", "contacts", "xyz", "json"),
)
```

Scientific / numerical caution

A bare DAT file cannot reconstruct Df, kf, seed, overlap model, or backend. Use bundle metadata or another explicit provenance record for scientific archival.

6.7 Interactive visualization

```
from fracval import ViewerAppearance, plot_3d

appearance = ViewerAppearance(
    particle_color="#CC5500",
    opacity=0.85,
    shininess=0.70,
    background_color="#FFFFFF",
    show_axes=False,
)

fig = plot_3d(agg, mode="spheres", appearance=appearance)
fig.show()
```

Radius coloring:

```
appearance = ViewerAppearance(
    color_mode="radius",
    colorscale="Plasma",
    show_colorbar=True,
    show_axes=False,
)

fig = plot_3d(agg, appearance=appearance)
```

For large N , `mode="centers"` is substantially lighter than generating sphere meshes.

6.8 Error handling

Configuration errors raise `ValueError`; generation failures raise `GenerationError`:

```
from fracval import GenerationError

try:
    agg = generate(cfg)
except (GenerationError, ValueError) as exc:
    print(f"FracVAL failed: {exc}")
```

Do not silently discard a generation failure in automated studies. Record the case parameters, seed, and error so the failed region can be investigated.

6.9 Diagnostics from Python

```
from fracval import format_runtime_info, runtime_info

print(format_runtime_info())
info = runtime_info()
```

This is useful when embedding FracVAL in another application and collecting a support report.

Chapter 7

Embedding FracVAL in Another Python Codebase

7.1 Recommended boundary

Treat FracVAL as a generator service returning data, not as a GUI component:

```
your_project/  
|-- simulation.py ---> fracval.FracVALConfig / fracval.generate  
|-- analysis.py   ---> Aggregate or NumPy arrays  
|-- database.py   ---> metadata / bundles  
'-- ui.py         ---> your own UI, if any
```

A minimal wrapper is:

Listing 7.1: Library-style integration example

```
"""A minimal integration pattern for another Python application."""  
from __future__ import annotations  
  
import numpy as np  
  
from fracval import FracVALConfig, generate  
  
def make_particle_cloud(  
    n: int,  
    df: float,  
    kf: float,  
    *,  
    seed: int | None = None,  
    polydispersity: float = 1.0,  
) -> np.ndarray:  
    """Return an ``(N, 4)`` array: x, y, z, radius.
```

This function is deliberately independent of the FracVAL GUI. It is the pattern to use when embedding FracVAL inside a solver, optimizer, notebook, database pipeline, or another desktop application.


```
"""
config = FracVALConfig(
    n=n,
    df=df,
    kf=kf,
    rp_g=15.0,
    rp_gstd=polydispersity,
    seed=seed,
)
return generate(config).data.copy()

if __name__ == "__main__":
    cloud = make_particle_cloud(100, 1.79, 1.40, seed=12345)
    print(cloud.shape)
    print(cloud[:5])
```

7.2 Why this boundary matters

The Python API is stable enough to use from notebooks, command-line programs, optimizers, and other desktop tools. In contrast, the Qt widget hierarchy is an implementation detail of the supplied GUI and can change without affecting scientific generation.

7.3 Parallel workloads

The legacy Fortran core stores runtime arrays and settings in module-global state. The F2PY backend therefore protects calls with a Python lock. The wrapper releases Python's GIL while Fortran executes, so the Qt event loop remains responsive, but native generation calls are serialized within one process.

For genuine parallel parameter sweeps, use separate processes rather than multiple Python threads. Each process has independent Fortran module state. Validate scaling and reproducibility on the target system before launching very large studies.

7.4 Adding Python-only post-processing

Analysis that does not affect construction should stay in Python. Example:

```
import numpy as np

def radial_distances(aggregate):
    center = aggregate.center_of_mass
    return np.linalg.norm(aggregate.xyz - center, axis=1)
```

This avoids touching the native engine and makes unit testing simpler.

Chapter 8

Standalone Fortran Runtime Interface

8.1 The namelist file

The executable reads a standard Fortran namelist at runtime. The default `fracval.in` is structurally:

```
&fracval
  N           = 100
  Df          = 1.79
  kf          = 1.40
  rp_g        = 15.0
  rp_gstd     = 1.00
  Quantity_aggregates = 1
  Ext_case    = 0
  Nsubcl_perc = 0.10
  tol_ov      = 1.0e-6
  random_seed_value = -1
  overlap_mode = 'none'
  overlap_fraction = 0.05
  overlap_mean = 0.05
  overlap_std  = 0.02
  overlap_max  = 0.12
  output_dir   = 'results'
/
```

The block starts with `&fracval` and ends with `/`. Fortran namelist names are case-insensitive. A relative output path is interpreted relative to the directory from which the executable is launched.

8.2 Input parameter reference

Parameter	Meaning	Validation / guidance
N	Primary particles per aggregate	integer, at least 5
Df	Fractal dimension	positive
kf	Fractal prefactor	positive
rp_g	Geometric mean primary-particle radius	positive

Parameter	Meaning	Validation / guidance
rp_gstd	Geometric standard deviation	at least 1; 1 is monodisperse
Quantity_aggregates	Number of successful aggregates to write	at least 1
Ext_case	Legacy extreme-case switch	0 or 1; normally 0
Nsubcl_perc	Initial sub-cluster fraction	in (0,1]; normally 0.10
tol_ov	Unintended-overlap numerical tolerance	positive; normally 10^{-6}
random_seed_value	Fortran random seed	-1 runtime default; nonnegative fixed seed
overlap_mode	Intended contact overlap	none, fixed, statistical
overlap_fraction	Fixed overlap fraction	0 means touching
overlap_mean	Statistical overlap mean	nonnegative fraction
overlap_std	Statistical overlap std. dev.	nonnegative
overlap_max	Statistical hard upper bound	in (0,0.95)
output_dir	Output directory	quoted path; created automatically

8.3 Run multiple cases without recompilation

```
./build/fracval cases/monodisperse.in
./build/fracval cases/polydisperse.in
./build/fracval cases/overlap_05.in
```

This is the central runtime-interface improvement: all normal simulation settings are read from the input file.

Chapter 9

Model and Numerical Behavior

9.1 Fractal target relation

The legacy placement routines compute target gyration radii using the geometric mean primary-particle radius and the fractal relation implemented in the source:

$$R_{g,\text{target}} = r_{p,g} \left(\frac{N}{k_f} \right)^{1/D_f}. \quad (9.1)$$

For polydisperse working sets, $r_{p,g}$ is obtained from the geometric mean of the radii currently being considered. This relation guides PCA/CCA placement; the Python post-generation `Aggregate.radius_of_gyration` is an independently calculated geometric metric of the final spheres.

9.2 Primary-particle size distribution

`rp_gstd=1.0` produces equal radii (monodisperse). Values above one use the lognormal primary-particle radius generator in the Fortran engine, centered on the geometric mean `rp_g`. Keep `rp_g`, coordinates, and all other lengths in one consistent unit system.

9.3 Construction restarts

For some stochastic configurations, a candidate PCA/CCA construction cannot be continued without violating constraints. The generator then restarts. This is normal behavior up to a point; repeated failure may indicate a difficult or incompatible parameter combination.

The Python F2PY interface limits this using `max_attempts`. The GUI exposes the same control. A generation that exhausts the limit raises `GenerationError` rather than hanging indefinitely.

Chapter 10

Intended Contact Overlap

10.1 Purpose

The overlap feature is a controlled model extension. It is not implemented by loosening global collision detection. Instead, it modifies only the selected particle-particle contact used to join a monomer or cluster.

For particles with radii R_i and R_j , touching contact has center distance

$$d_{ij} = R_i + R_j. \quad (10.1)$$

With intended overlap fraction ϵ , the joining distance is

$$d_{ij} = (R_i + R_j)(1 - \epsilon). \quad (10.2)$$

Thus $\epsilon = 0.05$ means a 5 percent reduction from touching separation, independent of the absolute particle size.

10.2 Fixed overlap

Python:

```
cfg = FracVALConfig(  
    n=100,  
    seed=12345,  
    overlap_mode="fixed",  
    overlap_fraction=0.05,  
)
```

Fortran namelist:

```
overlap_mode      = 'fixed'  
overlap_fraction = 0.05
```

10.3 Statistical overlap

For each intended joining contact, a normal candidate centered at `overlap_mean` with standard deviation `overlap_std` is sampled. Values outside `[0, overlap_max]` are rejected and redrawn. Therefore the realized contact distribution is a truncated/bounded distribution and its sample mean need not equal the unbounded input mean exactly.

```
cfg = FracVALConfig(  
    n=100,  
    seed=12345,  
    overlap_mode="statistical",  
    overlap_mean=0.05,  
    overlap_std=0.02,  
    overlap_max=0.12,  
)
```

10.4 Collision protection

All other newly introduced pair intersections remain subject to `tol_ov`. This is critical: enabling 5 percent intended overlap does not permit every particle pair in the aggregate to interpenetrate by 5 percent.

10.5 Recorded contact history

A successful tree-like aggregate normally contains $N - 1$ intended joining contacts. The Python object exposes:

```
agg.contact_overlaps  
agg.contact_count  
agg.mean_contact_overlap  
agg.std_contact_overlap  
agg.max_contact_overlap
```

The standalone executable writes a companion `.contacts.csv` file. Currently the contact sidecar records construction order and overlap fraction, not persistent final particle-pair IDs.

Scientific / numerical caution

Overlap changes the particle-center geometry and can therefore change derived morphology such as radius of gyration. Treat overlap settings as part of the physical model, not merely a rendering choice. Large overlap can also increase construction difficulty and restart frequency.

Chapter 11

Output Formats and Provenance

11.1 Native DAT

Each line contains:

```
x y z radius
```

There is no header, preserving compatibility with the legacy FracVAL data format.

11.2 CSV

The Python/GUI CSV export adds a header:

```
x,y,z,radius
```

11.3 Contact CSV

```
contact_index,overlap_fraction  
1,0.0471  
2,0.0618  
...
```

11.4 XYZ

The Python exporter uses a generic XYZ-like particle-center format with pseudo atom label P. Radius is not part of standard XYZ coordinates, so use DAT or CSV when particle size must be preserved.

11.5 Metadata JSON

`Aggregate.metadata()` includes the configuration, seed, attempts, backend, derived geometry, and contact-overlap array. This is the preferred machine-readable provenance record.

11.6 Interactive HTML

The GUI and Plotly helpers can export a self-contained interactive HTML view. This is a visualization artifact, not the authoritative scientific data file.

Chapter 12

Visualization Tools

12.1 Standalone plotting script

Plot a saved aggregate:

```
python plot/plot_aggregate.py results/N_00000100_Agg_00000001.dat
```

Customize:

```
python plot/plot_aggregate.py aggregate.dat \  
  --color '#CC5500' \  
  --opacity 0.85 \  
  --shininess 0.75 \  
  --background '#F5F5F5' \  
  --no-title \  
  --output aggregate.html
```

XYZ axes are hidden by default. Add `-axes` when coordinate orientation is important.

12.2 Notebooks

`plot/plot_aggregate.ipynb` demonstrates visualization of existing files. `plot/generate_and_visualize.ipynb` generates directly through the Python API and plots the returned aggregate.

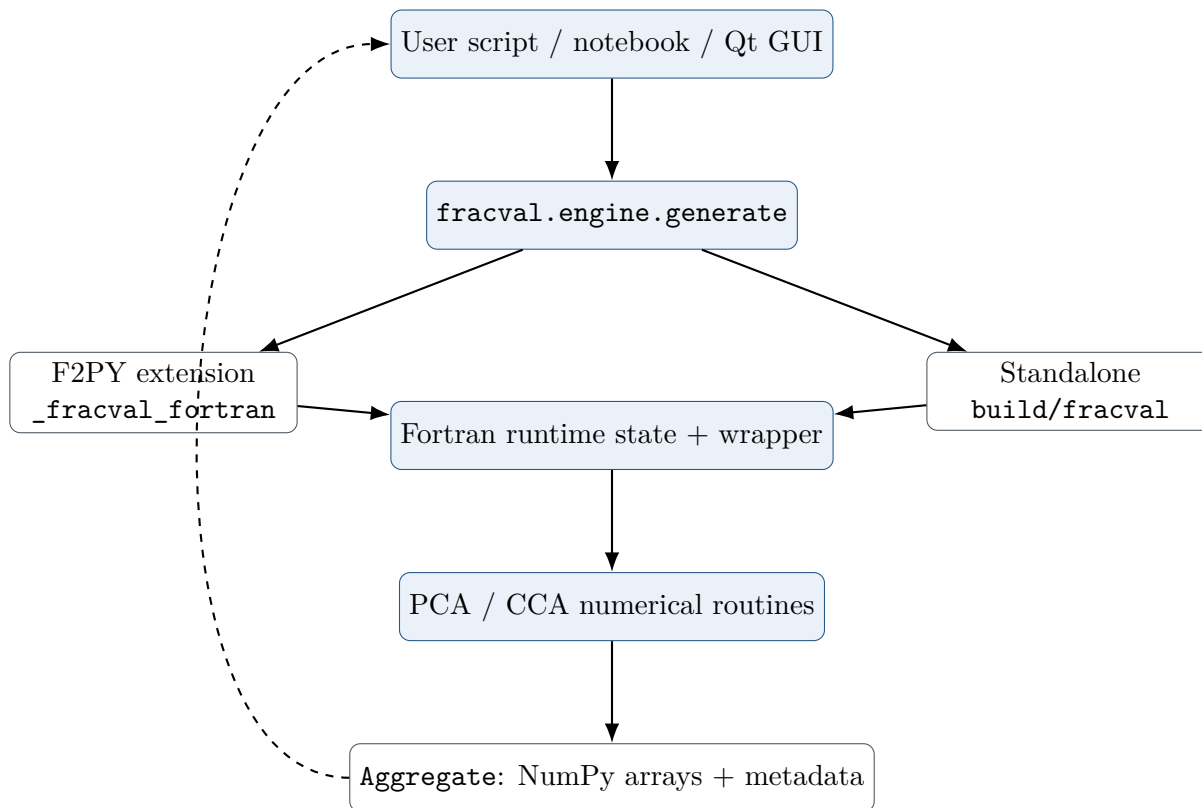
12.3 Performance considerations

Sphere meshes create many vertices and triangles. For very large aggregates, switch to center-marker mode for interactive inspection and use high-resolution sphere rendering only for selected output figures.

Chapter 13

Backends and Code Architecture

13.1 Call graph



13.2 F2PY extension backend

The preferred backend calls `fracval_generate` in `src/fracval_python_api.f90`. Inputs are scalar runtime values; outputs are x, y, z, r arrays plus realized contact overlaps and status information.

The wrapper is marked thread-safe at the F2PY boundary so the generated C layer releases the Python GIL while native Fortran runs. However, the underlying Fortran modules use shared global state, so `engine.py` still serializes extension calls with a process-local lock.

13.3 Executable fallback

The executable backend writes a temporary namelist, invokes the standalone program, reads the produced DAT/contact files, and deletes the temporary working directory. A custom executable may be supplied to `generate()` or through the `FRACVAL_EXECUTABLE` environment variable.

13.4 Why both backends exist

The extension is faster and cleaner for Python because it avoids temporary files. The executable fallback is valuable for debugging, cross-checking, and environments where building a Python extension is inconvenient. Regression tests compare the two for fixed-seed cases.

Chapter 14

Testing and Validation

14.1 Core test command

```
python -m pytest
```

This runs standalone Fortran cases plus Python/F2PY regression checks. `tests/python/test_fortran_cli.py` replaced the earlier shell harness (`tests/run_tests.sh`), so the whole suite runs under `pytest` on every platform.

14.2 What the tests cover

The current suite checks:

- runtime namelist execution;
- monodisperse radius behavior;
- polydisperse radius variation;
- fixed-seed F2PY reproducibility;
- F2PY versus standalone executable equivalence within output precision;
- $N - 1$ zero contact records in legacy no-overlap mode;
- fixed 5 percent overlap at intended contacts;
- bounded and varying statistical overlap;
- final pair geometry to detect unexpected overlapping pairs;
- visualization construction;
- configuration JSON and result-bundle round trips;
- runtime backend diagnostics.

14.3 Qt tests

After installing GUI dependencies:

```
make qt-check PYTHON=python
make gui-test PYTHON=python
```

or directly with pytest:

```
FRACVAL_RUN_GUI_TESTS=1 python -m pytest tests/python/test_qt_gui.py
```

The GUI smoke test selects an available headless Qt platform plugin and constructs the application without requiring a visible display. It is opt-in and is not collected by a plain `python -m pytest` run.

14.4 Debug Fortran build

```
make debug
```

or, on any platform:

```
python tools/build.py all --debug
```

This rebuilds with warnings, bounds/runtime checks, debug symbols, and backtraces. Use it when changing numerical code.

Chapter 15

Extending FracVAL Safely

15.1 Python-only feature

If a feature only analyzes, filters, visualizes, or exports an already-created aggregate, implement it in Python. Add tests that create an **Aggregate** or use a fixed-seed generated sample. There is no need to change the Fortran core.

15.2 Adding a new physical generation parameter

A setting that affects placement should be propagated consistently:

1. add runtime storage/default/validation to `src/Ctes.f90`;
2. modify the appropriate PCA/CCA routine;
3. add the parameter to `src/fracval_python_api.f90`;
4. update `python/fracval/_fracval_fortran.pyf`;
5. add a field and validation to **FracVALConfig**;
6. pass it from `engine.py`;
7. include it in executable namelist generation;
8. add GUI controls only after the API path works;
9. add fixed-seed tests and update documentation.

Important

Do not implement scientific behavior exclusively in the GUI. The GUI should be a client of the same generation API used by scripts and tests.

15.3 Adding a new export format

If it is a text/data format, add a conversion method to **Aggregate** or a helper in `io.py`, then expose it through the GUI only as a thin file-dialog layer. Keep metadata/provenance explicit.

15.4 Adding another visualization backend

Keep **Aggregate** as the input boundary. A future VTK/PyVista/OpenGL viewer can replace or complement Plotly without changing the Fortran or generation API.

15.5 Version-control guidance

Do not commit platform-specific extension binaries, object files, module files, or local virtual environments. The supplied **make clean** target removes native build outputs. Keep fixed-seed regression cases in version control so scientific changes can be compared deliberately.

Chapter 16

Troubleshooting

16.1 No backend is available

Run:

```
fracval-info
```

Then build one or both backends:

```
make
make python-ext PYTHON=python
```

16.2 Could not find the Qt platform plugin "cocoa"

First ensure the same environment is used for install and launch:

```
source .venv/bin/activate
which python
python -c "import PySide6; print(PySide6.__file__)"
fracval-qt-check
```

The runtime helper searches the active PySide6 installation and common Conda Qt plugin layouts, then configures the plugin directory before Qt starts.

If PySide6 itself is incomplete, reinstall it in the active environment:

```
python -m pip uninstall -y PySide6 PySide6_Addons PySide6_Essentials shiboken6
python -m pip install --no-cache-dir PySide6
```

16.3 F2PY extension build fails

Check:

```
which python
```



```
python -c "import numpy; print(numpy.__version__)"
fracval-info
```

`fracval-info` reports the discovered Fortran and C compilers, so a missing or mismatched toolchain shows up directly. Then clean and rebuild:

```
python tools/build.py clean
python tools/build.py ext
```

Because the extension ABI is Python/platform-specific, rebuild it after changing to a different Python installation or architecture.

16.4 Windows: ImportError: DLL load failed for the extension

The extension is linked with a static GNU runtime and verified at build time. If an import still fails, rebuild with `python tools/build.py ext` from the activated `fracval` environment; the build script copies `libgfortran-5.dll`, `libquadmath-0.dll`, `libgcc_s_seh-1.dll`, and `libwinpthread-1.dll` beside the extension when they are needed.

16.5 Many PC not able to continue / restart messages

A restart means the stochastic algorithm could not continue the current construction while satisfying its geometry checks. Possible responses:

- verify that `Df`, `kf`, size distribution, and overlap are reasonable for the intended model;
- increase `max_attempts` for a legitimately difficult case;
- keep `Nsubcl_perc=0.10` unless a known legacy case requires adjustment;
- reduce aggressive intended overlap and retest;
- run a debug build if behavior changed after code modifications.

Do not simply relax `tol_ov` to force a physical overlap model.

16.6 The GUI opens but generation fails

Check the status/error dialog, then run the same configuration through Python with a fixed seed. Scripted generation produces a more convenient exception for logging and is easier to reproduce in a bug report.

16.7 3-D rendering is slow

Switch Display from Spheres to Centers or lower sphere resolution. Rendering cost is separate from Fortran generation cost.

Chapter 17

Command and API Reference

17.1 Installed commands

Command	Purpose
<code>fracval-gui</code>	Launch native PySide6 desktop application.
<code>fracval-python</code>	Generate one aggregate from command-line Python arguments.
<code>fracval-info</code>	Report Python, package, extension and backend diagnostics.
<code>fracval-qt-check</code>	Report Qt/PySide6 plugin paths and platform availability.
<code>python tools/build.py</code>	Cross-platform build script (<code>exe</code> , <code>ext</code> , <code>all</code> , <code>clean</code>).

17.2 Public Python functions

Name	Purpose
<code>generate(config, ...)</code>	Generate one aggregate.
<code>generate_batch(count, config, ...)</code>	Return a list of aggregates.
<code>iter_generate_batch(count, config, ...)</code>	Stream aggregates one at a time.
<code>available_backends()</code>	List currently usable generation backends.
<code>extension_available()</code>	Test whether the F2PY extension imports.
<code>load_data(path)</code>	Read a four-column native DAT file into NumPy.
<code>save_bundle(aggregate, directory, ...)</code>	Save data, contacts, and metadata with a common stem.
<code>load_bundle(metadata_path)</code>	Reconstruct an Aggregate from a saved bundle.
<code>plot_3d(...)</code>	Return an interactive Plotly 3-D figure.
<code>plot_static(...)</code>	Return a Matplotlib 3-D figure.

`runtime_info()`

Return environment diagnostics as a dictionary.

17.3 FracVALConfig fields

Field	Default	Meaning
<code>n</code>	100	primary-particle count
<code>df</code>	1.79	fractal dimension
<code>kf</code>	1.40	fractal prefactor
<code>rp_g</code>	15.0	geometric mean radius
<code>rp_gstd</code>	1.0	geometric radius standard deviation
<code>ext_case</code>	0	legacy extreme-case flag
<code>nsubcl_perc</code>	0.10	sub-cluster fraction
<code>tol_ov</code>	10^{-6}	unintended-overlap numerical tolerance
<code>seed</code>	None	fixed seed or Python-selected fresh seed
<code>max_attempts</code>	250	maximum construction restarts in extension path
<code>overlap_mode</code>	none	none, fixed, or statistical
<code>overlap_fraction</code>	0.0	fixed intended-contact overlap
<code>overlap_mean</code>	0.05	statistical mean
<code>overlap_std</code>	0.02	statistical standard deviation
<code>overlap_max</code>	0.12	statistical hard upper bound

17.4 ViewerAppearance fields

Field	Default	Meaning
<code>color_mode</code>	solid	solid or radius-based coloring
<code>particle_color</code>	#4C78A8	solid particle color
<code>colorscale</code>	Viridis	Plotly radius color scale
<code>opacity</code>	0.96	particle opacity
<code>shininess</code>	0.55	specular/gloss appearance in sphere mode
<code>background_color</code>	#FFFFFF	scene/page background
<code>show_axes</code>	false	show XYZ axes/grid
<code>show_colorbar</code>	false	show radius legend when radius-colored
<code>show_title</code>	true	show figure title

Chapter 18

Worked Python Examples

The following files are included under `examples/python/`. They are meant to be copied and adapted rather than treated as hidden framework code.

18.1 Basic generation and bundle export

```
"""Generate one reproducible monodisperse aggregate."""
from fracval import FracVALConfig, generate, save_bundle

config = FracVALConfig(
    n=100,
    df=1.79,
    kf=1.40,
    rp_g=15.0,
    rp_gstd=1.0,
    seed=12345,
)

aggregate = generate(config)
print(f"backend: {aggregate.backend}")
print(f"seed: {aggregate.seed}")
print(f"particles: {aggregate.n}")
print(f"radius of gyration: {aggregate.radius_of_gyration:.6g}")
print(f"bounding radius: {aggregate.bounding_radius:.6g}")

written = save_bundle(aggregate, "example_output/basic", stem="monodisperse")
for kind, path in written.items():
    print(f"{kind}: {path}")
```

18.2 Overlap comparison

```
"""Compare the intended-contact overlap modes."""
from fracval import FracVALConfig, generate

base = FracVALConfig(n=50, seed=24680, max_attempts=500)
```

```

cases = {
    "none": base.with_updates(overlap_mode="none"),
    "fixed_5_percent": base.with_updates(
        overlap_mode="fixed", overlap_fraction=0.05
    ),
    "statistical": base.with_updates(
        overlap_mode="statistical",
        overlap_mean=0.05,
        overlap_std=0.02,
        overlap_max=0.12,
    ),
}

for name, config in cases.items():
    aggregate = generate(config)
    print(
        f"{name:16s} contacts={aggregate.contact_count:3d} "
        f"mean={100*aggregate.mean_contact_overlap:6.3f}% "
        f"std={100*aggregate.std_contact_overlap:6.3f}% "
        f"max={100*aggregate.max_contact_overlap:6.3f}%"
    )

```

18.3 Batch generation

```

"""Generate a deterministic batch and save each aggregate as a bundle."""
from pathlib import Path

from fracval import FracVALConfig, iter_generate_batch, save_bundle

output = Path("example_output/batch")
config = FracVALConfig(n=100, df=1.79, kf=1.40, seed=20260902)

for index, aggregate in enumerate(iter_generate_batch(10, config), start=1):
    stem = f"aggregate_{index:04d}_seed_{aggregate.seed}"
    save_bundle(aggregate, output, stem=stem)
    print(index, aggregate.seed, aggregate.radius_of_gyration)

```

18.4 Parameter sweep

```

"""Small parameter sweep suitable as a template for a research workflow."""
import csv
from pathlib import Path

from fracval import FracVALConfig, generate

rows = []
base_seed = 10000
for i, df in enumerate((1.6, 1.8, 2.0)):
    for j, kf in enumerate((0.9, 1.2, 1.5)):

```

```
    config = FracVALConfig(
        n=100,
        df=df,
        kf=kf,
        rp_g=15.0,
        rp_gstd=1.0,
        seed=base_seed + 100 * i + j,
        max_attempts=500,
    )
    aggregate = generate(config)
    rows.append({
        "df": df,
        "kf": kf,
        "seed": aggregate.seed,
        "radius_of_gyration": aggregate.radius_of_gyration,
        "bounding_radius": aggregate.bounding_radius,
        "attempts": aggregate.attempts,
    })
    print(rows[-1])

out = Path("example_output/parameter_sweep.csv")
out.parent.mkdir(parents=True, exist_ok=True)
with out.open("w", newline="") as handle:
    writer = csv.DictWriter(handle, fieldnames=rows[0])
    writer.writeheader()
    writer.writerows(rows)
print(f"wrote {out}")
```

Appendix A

Makefile Target Reference

Target	Action
<code>make</code>	Build <code>build/fracval</code> .
<code>make run</code>	Run default <code>fracval.in</code> .
<code>make run INPUT=x</code>	Run a named input file.
<code>make install</code>	Install editable Python package and F2PY extension.
<code>make install-gui</code>	Install GUI dependencies and extension.
<code>make info</code>	Print Python/backend diagnostics.
<code>make python-ext</code>	Build in-memory Fortran extension.
<code>make fortran-test</code>	Run standalone cases.
<code>make python-test</code>	Run Python scientific/visualization tests.
<code>make test</code>	Run the pytest suite.
<code>make qt-check</code>	Diagnose Qt platform plugins.
<code>make gui-test</code>	Headless Qt construction smoke test.
<code>make gui</code>	Launch the native desktop application.
<code>make plot-test</code>	Exercise static and interactive plotting.
<code>make debug</code>	Rebuild Fortran with runtime checking.
<code>make docs</code>	Compile this LaTeX manual to PDF.
<code>make clean</code>	Remove native build artifacts (<code>tools/build.py clean</code>).
<code>python tools/build.py all</code>	Build executable and extension on any platform.

Appendix B

Recommended Scientific Workflow Checklist

1. Record `N`, `Df`, `kf`, particle-size distribution, overlap model, and seed.
2. Use a fixed seed while developing or validating a case.
3. Run `make test` after modifying the Fortran/Python interface.
4. Save aggregate metadata or a bundle, not only a DAT file.
5. Keep overlap distinct from `tol_ov`.
6. Use the F2PY backend for normal Python work and the executable backend as a useful cross-check.
7. For parameter sweeps, log failed seeds/configurations instead of silently skipping them.
8. Use multiprocessing rather than threads for process-level generation parallelism.
9. Rebuild the native extension after changing Python versions or architectures.
10. Keep the GUI as an interface layer; add reusable scientific functionality to the public Python API.

Recommended practice

The fastest route for a new user is: create a virtual environment, run `make install-gui PYTHON=python`, run `make`, verify with `fracval-info`, and then use either `fracval-gui` or the public `fracval` Python API. The full project is designed so those two workflows share the same Fortran generator.